### 4.6.1 Fibonacci Series – An example

Fibonacci series generates the subsequent number by adding two previous numbers. Fibonacci series starts from two numbers – Fib 0 & Fib 1. The initial values of Fib 0 & Fib 1 can be taken as 0 and 1.

Fibonacci series satisfies the following conditions :

$Fibn = Fib_{n-1} + Fib_{n-2}$

Hence, a Fibonacci series for the n value 8 can look like this

$Fib_8$ = 0 1 1 2 3 5 8 13

### 4.6.2 Fibonacci Iterative Algorithm with Dynamic programming approach

The following example shows a simple Dynamic programming approach for the generation of Fibonacci series.

Initialize f0=0, f1 =1

step-1: Print the initial values of Fibonacci f0 and f1

step-2: Calculate fibanocci fib ← f0 + f1

step-3: Assign f0← f1, f1← fib

step-4: Print the next consecutive value of fibanocci fib

step-5: Goto step-2 and repeat until the specified number of terms generated

For example if we generate fibobnacci series upto 10 digits, the algorithm will generate the series as shown below:

The Fibonacci series is : 0 1 1 2 3 5 8 13 21 34 55

**Points to remember:**

- An algorithm is a finite set of instructions to accomplish a particular task.
- Algorithm consists of step-step-by instructions that are required to accomplish a task and helps the programmer to develop the program.
- Program is an expression of algorithm in a programming language.
- Algorithm analysis deals with the execution or running time of various operations involved.
- Space complexity of an algorithm is the amount of memory required to run to its completion.
- Big Oh is often used to describe the worst-case of an algorithm.
- Big Omega is used to describe the lower bound which is best way to solve the space complexity.
- The Time complexity of an algorithm is given by the number of steps taken by the algorithm to complete the process.
- The efficiency of an algorithm is defined as the number of computational resources used by the algorithm.